

A Robust Local Review Agent for Fast, Evidence-Grounded Paper Assessment

A Robust Local Review Agent for Fast, Evidence-Grounded Paper Assessment

Abstract

We present a local Review Agent designed for the Ralphthon ICML 2026 Track 2 technical-report submission. The system accepts assigned paper PDFs, extracts their text, evaluates each paper with a structured ICML-style rubric, detects reviewer-directed prompt injection, computes the overall recommendation deterministically, and exports a reproducible JSON, Markdown, and PDF review bundle. The implementation is optimized for a short review window: PDF parsing is local, independent papers can be processed concurrently, and all model calls use a local Ollama Gemma model so no cloud API key or paper content leaves the machine. The design prioritizes calibrated failure handling over confident but unsupported scores.

1. Problem and Design Goals

Track 2 requires an agent that reviews papers produced by Track 1. The practical constraints are a limited time window, multiple PDF inputs, possible malformed model outputs, and adversarial text inside papers. We therefore optimize for five properties: throughput, evidence grounding, deterministic scoring, injection resistance, and recoverability.

2. System Overview

The pipeline has four stages. First, pypdf extracts text from each assigned PDF. Second, a local Gemma model produces independent scores for soundness, novelty, clarity, and significance together with strengths, weaknesses, questions, and an injection judgment. Third, a deterministic rubric computes the weighted overall score and recommendation; the model is not trusted to invent the final decision. Finally, the system writes `batch_reviews.json`, `batch_reviews.md`, and `batch_reviews.pdf`.

The reviewer treats each paper as data rather than instructions. A heuristic scan flags phrases such as requests to ignore instructions, give a perfect score, or accept the paper. The flag is included in the review evidence and caps the computed score, preventing hidden reviewer manipulation from being rewarded.

3. Throughput and Local Execution

The batch runner uses bounded concurrency so independent papers can be reviewed in parallel without creating an unbounded memory or GPU queue. PDF extraction and report generation are local. The default model is `gemma4:e4b-mlx` through Ollama, selected because it is already installed on the Apple Silicon machine and does not require an API key. Structured output is requested as JSON; fenced JSON, repaired JSON, and common list-wrapped objects are normalized before validation.

The system is deliberately conservative about concurrency. A small number of workers is preferable to launching multiple model copies: the model stays warm in memory while requests are bounded. This gives the reviewer a predictable latency profile and avoids wasting the review window on model reloads.

4. Scoring Rubric

The four sub-scores are independently reported on a 1–10 scale. The overall score is computed as:

$$0.35 \text{ soundness} + 0.30 \text{ novelty} + 0.15 \text{ clarity} + 0.20 \text{ significance}$$

The recommendation thresholds are Accept at 7.5 or higher, Weak Accept at 6.0, Borderline at 4.5, and Reject below 4.5. If reviewer-directed injection is detected, the overall score is capped at 3.0. Missing or malformed local-model fields receive conservative fallback values and are explicitly marked in the stored evidence.

5. Reproducibility and Artifacts

The repository contains the complete runner, prompts, rubric, and PDF renderer. A batch invocation is:

```
python review_batch.py papers --workers 5
```

The result bundle preserves one review per input filename, the calculated recommendation, strengths, weaknesses, injection evidence, and any extraction or model error. The PDF report is generated from the same Markdown summary, ensuring that the human-readable submission and machine-readable record are aligned.

6. Limitations

This system does not claim that local-model judgments equal expert human judgments. PDF text extraction can lose figures, equations, and layout-dependent evidence. The current injection detector is heuristic and should be supplemented by human review. A full benchmark across ten organizer-provided papers was not available during development, so throughput and calibration should be measured again on the actual assignment.

7. Conclusion

The resulting Review Agent is a compact, auditable system for reviewing multiple papers under time pressure. Its contribution is not an unsupported claim of superior reviewing; it is a reproducible workflow that makes its evidence, uncertainty, security flags, and deterministic scoring visible to judges.